

FIAP — FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

GUSTAVO DE JESUS SILVA — RM567926
GUSTAVO RODRIGUES SICILIANO — RM568419
SAMUEL KENITI KINA DE LIMA — RM567614

AGROSAT — MONITORAMENTO E RISCO AGRÍCOLA VIA SATÉLITE
DOMAIN DRIVEN DESIGN USING JAVA

São Paulo
2026

GUSTAVO DE JESUS SILVA — RM567926
GUSTAVO RODRIGUES SICILIANO — RM568419
SAMUEL KENITI KINA DE LIMA — RM567614

AGROSAT — MONITORAMENTO E RISCO AGRÍCOLA VIA SATÉLITE
DOMAIN DRIVEN DESIGN USING JAVA

Global Solution apresentada ao
curso de Análise e
Desenvolvimento de Sistemas da
FIAP — Faculdade de
Tecnologia, como requisito
parcial para a obtenção de nota.

Orientador: Fernando Luiz de
Almeida

São Paulo
2026

SUMÁRIO

SUMÁRIO	3
1. INTRODUÇÃO	4
2. OBJETIVO E ESCOPO	5
3. DESCRIÇÃO DA SOLUÇÃO	6
4. ESTRUTURA DO PROJETO	7
5. FUNCIONALIDADES PRINCIPAIS	8
5.1 Cadastro de Regiões com Automação Geográfica	8
5.2 Leitura Satelital Automática	8
5.3 Integração com IA no Render	8
5.4 Tratamento de Exceções	8
5.5 CORS	8
6. TELAS DO SISTEMA	10
6.1 Painel de Alertas — GET /alertas e GET /alertas/pendentes	10
6.2 Correlação entre a API e o Frontend	10
6.3 Observação sobre as demais telas	11
7. TABELA DE ENDPOINTS	15
7.1 Health Check	15
7.2 Fontes Satelitais	15
7.3 Regiões	15
7.4 Usuários	15
7.5 Leituras Satelitais	15
7.6 Previsões	16
7.7 Alertas	16
7.8 Regiões × Usuários	16
8. INSTRUÇÕES DE EXECUÇÃO	17
8.1 Execução Local	17
8.2 MainTeste — Terminal Interativo	17
8.3 Deploy no Render	17
9. FERRAMENTAS E VERSÕES	18
10. LINKS DO PROJETO	19
11. CONCLUSÃO	20

1. INTRODUÇÃO

O AgroSat é uma plataforma de monitoramento agrícola via satélite com Inteligência Artificial, desenvolvida para democratizar o acesso à tecnologia espacial entre pequenos e médios produtores rurais brasileiros. Este documento descreve a camada de backend da solução. Uma API REST desenvolvida com o framework Quarkus, integrada ao banco de dados Oracle e a APIs externas de dados satelitais.

GitHub: <https://github.com/GustavoSiciliano/Global-Solution-2026>

Deploy: <https://global-solution-2026.onrender.com>

Vídeo Demonstração: <https://youtu.be/tiokmaJ4eYg>

2. OBJETIVO E ESCOPO

O objetivo desta entrega é uma API RESTful completa construída com Quarkus 3.8.4 e Java 21, que expõe todos os dados e operações necessários para o funcionamento do frontend React do AgroSat. A API é responsável por:

- Persistir regiões agrícolas monitoradas no Oracle FIAP
- Coletar dados satelitais reais via NASA POWER e Open-Meteo automaticamente
- Consultar coordenadas e bioma via ViaCEP, Nominatim e INPE
- Enviar dados ao modelo de Machine Learning hospedado no Render
- Gerar previsões de risco hídrico e alertas automáticos quando risco $\geq 40\%$
- Gerenciar usuários, fontes satelitais e vínculos região-usuário com CRUD completo

O escopo inclui todas as 8 tabelas do banco de dados Oracle: TB_FONTE_SATELITAL, TB_REGIAO, TB_USUARIO, TB_REGIAO_USUARIO, TB_LEITURA_SATELITAL, TB_PREVISAO, TB_ALERTA e TB_HISTORICO_ALERTA.

3. DESCRIÇÃO DA SOLUÇÃO

A API AgroSat Java segue o padrão arquitetural DAO/BO/Resource ensinado em aula, com camadas bem definidas de responsabilidade. Toda comunicação com o Oracle utiliza JDBC puro com gerenciamento explícito de transações (setAutoCommit, commit e rollback). O tratamento de exceções é centralizado com retorno de JSON padronizado para o frontend.

A principal automação do sistema é acionada ao salvar uma leitura satelital: a API coleta dados reais da NASA POWER e Open-Meteo, estima o NDVI com base no bioma da região, envia ao modelo Flask IA no Render e salva automaticamente a previsão gerada. Se o risco hídrico retornado for maior ou igual a 40%, um alerta é criado automaticamente na tabela TB_ALERTA.

4. ESTRUTURA DO PROJETO

O projeto segue a estrutura de pacotes abaixo, organizada por responsabilidade:

Pacote	Descrição
AgroSat.vo	Value Objects — classes com getters e setters representando as tabelas do banco (Regiao, Usuario, LeituraSatelital, Previsao, Alerta, HistoricoAlerta, FonteSatelital, RegiaoUsuario, ResultadoIA, SituacaoRegiao)
AgroSat.conexao	Conexão Oracle (ConexaoBanco) e clientes das APIs externas: ApiViaCep, ApiNominatim, ApiInpe, ApiNasaPower, ApiOpenMeteo
AgroSat.dao	Data Access Objects — todas as operações SQL com o Oracle (8 DAOs, um por tabela)
AgroSat.bo	Business Objects — regras de negócio e validações (8 BOs com validações completas)
AgroSat.render	RenderIA — cliente HTTP que chama o modelo Flask IA hospedado no Render
AgroSat.api	Endpoints REST — classes JAX-RS com todos os verbos HTTP (8 classes de recurso)
AgroSat.excecao	Tratamento de exceções — Excecao (RuntimeException com código HTTP) e ManipuladorExcecao (@Provider que converte para JSON)
AgroSat.Teste	MainTeste.java — cliente de teste interativo via terminal

5. FUNCIONALIDADES PRINCIPAIS

5.1 Cadastro de Regiões com Automação Geográfica

O usuário informa apenas a cidade e o estado. O sistema chama automaticamente:

- ViaCEP — converte CEP em cidade e estado
- Nominatim (OpenStreetMap) — converte cidade/estado em latitude e longitude
- INPE TerraBrasilis — identifica o bioma pelas coordenadas
- Fallback: se o bioma não for identificado (cidades urbanas), registra como 'Nao identificado'

5.2 Leitura Satelital Automática

- NASA POWER — temperatura média e radiação solar dos últimos 7 dias
- Open-Meteo — precipitação, umidade e dias consecutivos sem chuva (últimos 30 dias)
- NDVI estimado por faixa do bioma da região

5.3 Integração com IA no Render

Após salvar a leitura, a API envia os dados ao modelo Flask IA. O modelo retorna status da vegetação, risco hídrico e nível do alerta. A previsão é salva automaticamente. Se o risco for $\geq 40\%$, um alerta é gerado em TB_ALERTA.

5.4 Tratamento de Exceções

Situação	Resposta JSON
Região não encontrada	{"erro": "Regiao nao encontrada: 999", "status": 404}
Estado inválido	{"erro": "Estado invalido: ZZ", "status": 400}
Email duplicado	{"erro": "Email ja cadastrado", "status": 409}
Erro interno	{"erro": "Erro ao inserir leitura: ...", "status": 500}

5.5 CORS

Propriedade	Valor
quarkus.http.cors	true
quarkus.http.cors.origins	/*
quarkus.http.cors.methods	GET, POST, PUT, DELETE, OPTIONS
quarkus.http.cors.headers	Content-Type, Authorization, Accept

6. TELAS DO SISTEMA

As imagens abaixo mostram o frontend React consumindo diretamente os dados fornecidos pela API Java. Cada elemento visual tem correspondência direta com um endpoint REST da API.

6.1 Painel de Alertas — GET /alertas e GET /alertas/pendentes

A tela de alertas exibe em tempo real os registros da tabela TB_ALERTA, gerados automaticamente pela API Java quando o modelo de IA retorna risco hídrico $\geq 40\%$. Cada card corresponde a um objeto JSON retornado pelo endpoint GET /alertas, contendo os campos dsNivel (CRÍTICO, ALTO, MEDIO, BAIXO), dsStatusVegetacao (SAUDAVEL, EM_ESTRESSE, CRÍTICO), dsMensagem, dtAlerta e flResolvido.

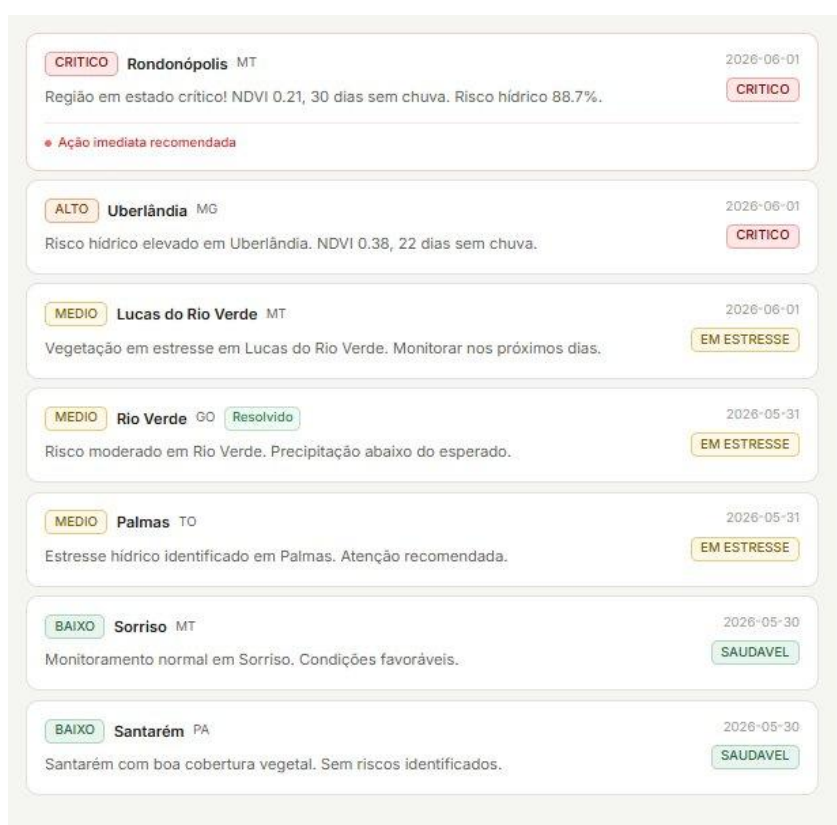


Figura 1 — Painel de alertas do AgroSat consumindo GET /alertas da API Java

6.2 Correlação entre a API e o Frontend

A tabela abaixo detalha a relação direta entre os endpoints da API Java e os elementos exibidos no frontend:

Endpoint Java	Dado retornado	Elemento no frontend
GET /alertas	Lista de alertas ordenada por criticidade	Cards de alerta com badge de nível
GET /alertas/pendentes	Apenas alertas com flResolvido = N	Contador de pendências no menu
PUT /alertas/id/resolver	Alerta com flResolvido = S	Badge 'Resolvido' no card
GET /regioes	Lista de regiões com bioma e coords	Mapa do Brasil e seletor de regiões
GET /previsoes/regiao/id}	Previsões de risco hídrico	Gráfico de risco e status de vegetação
POST /leituras/automatico/id}	Leitura + previsão geradas	Dashboard com dados atualizados

6.3 Observação sobre as demais telas

As telas abaixo mostram o Dashboard principal, a tela de Clima em tempo real e o Comparativo de Regiões, todas consumindo diretamente os endpoints da API Java hospedada no Render.

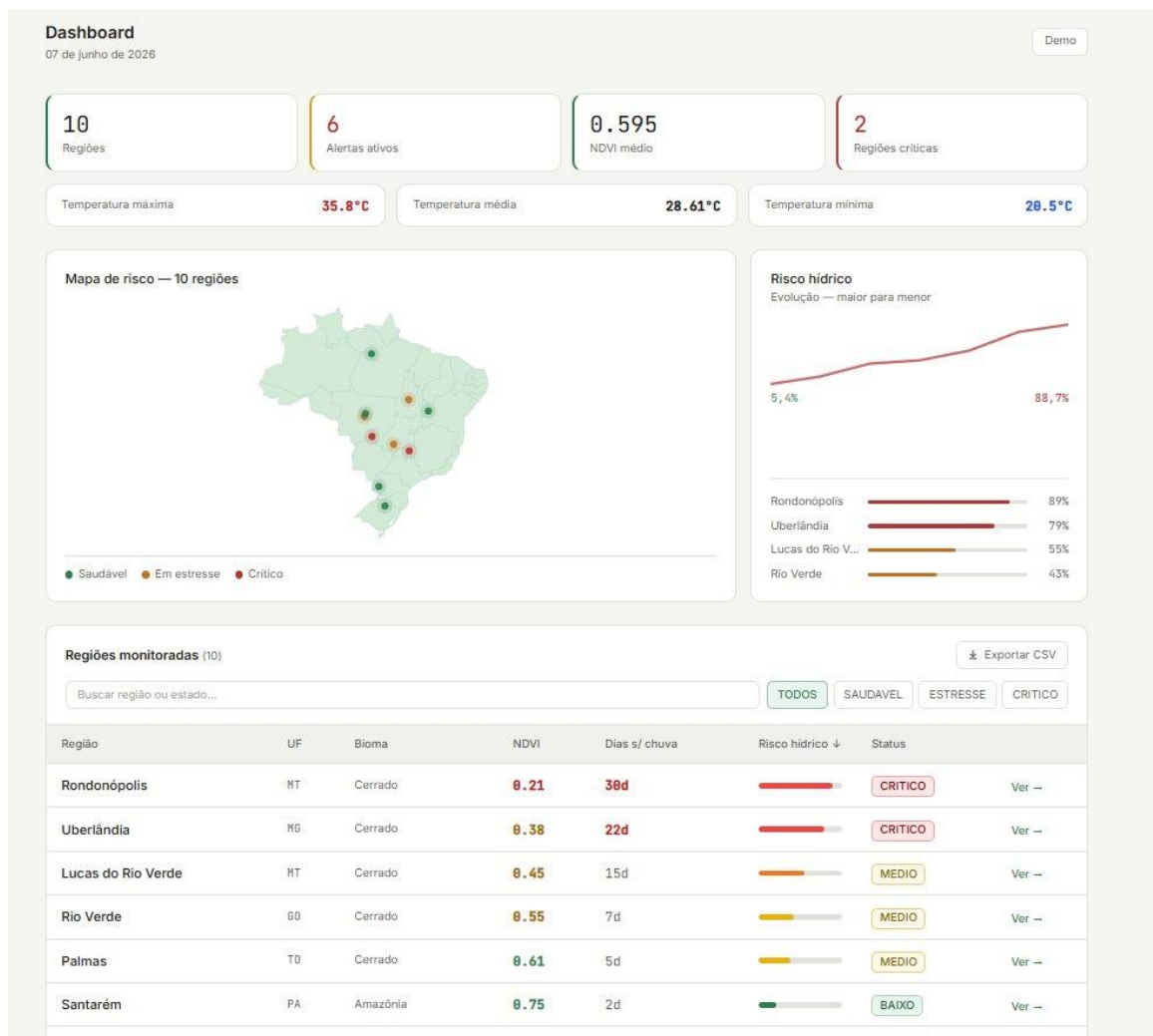


Figura 2 — Dashboard principal: GET /regioes, GET /previsoes e GET /alertas/pendentes

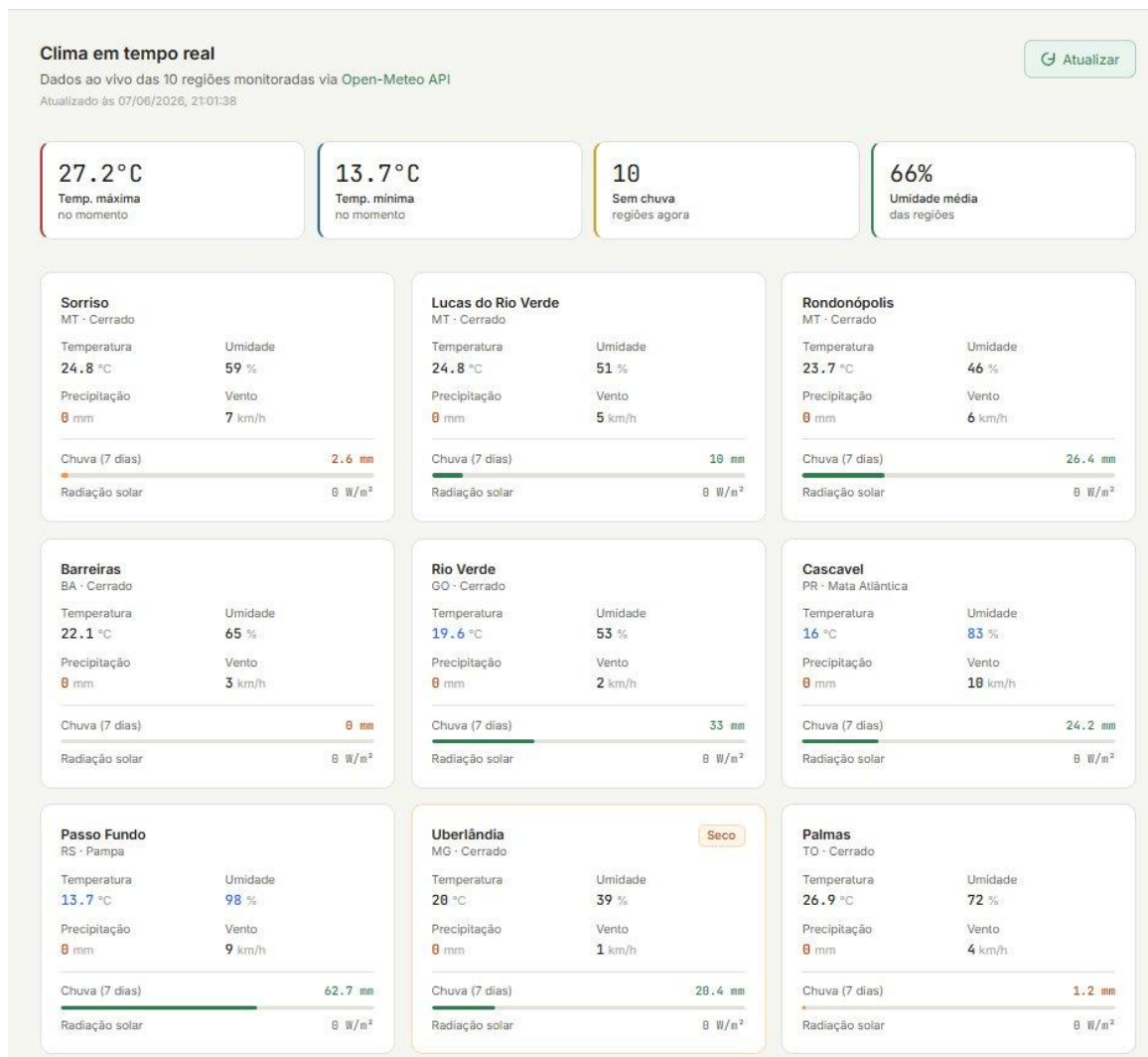


Figura 3 — Clima em tempo real: GET /leituras/regiao/{id} com dados da NASA POWER e Open-Meteo

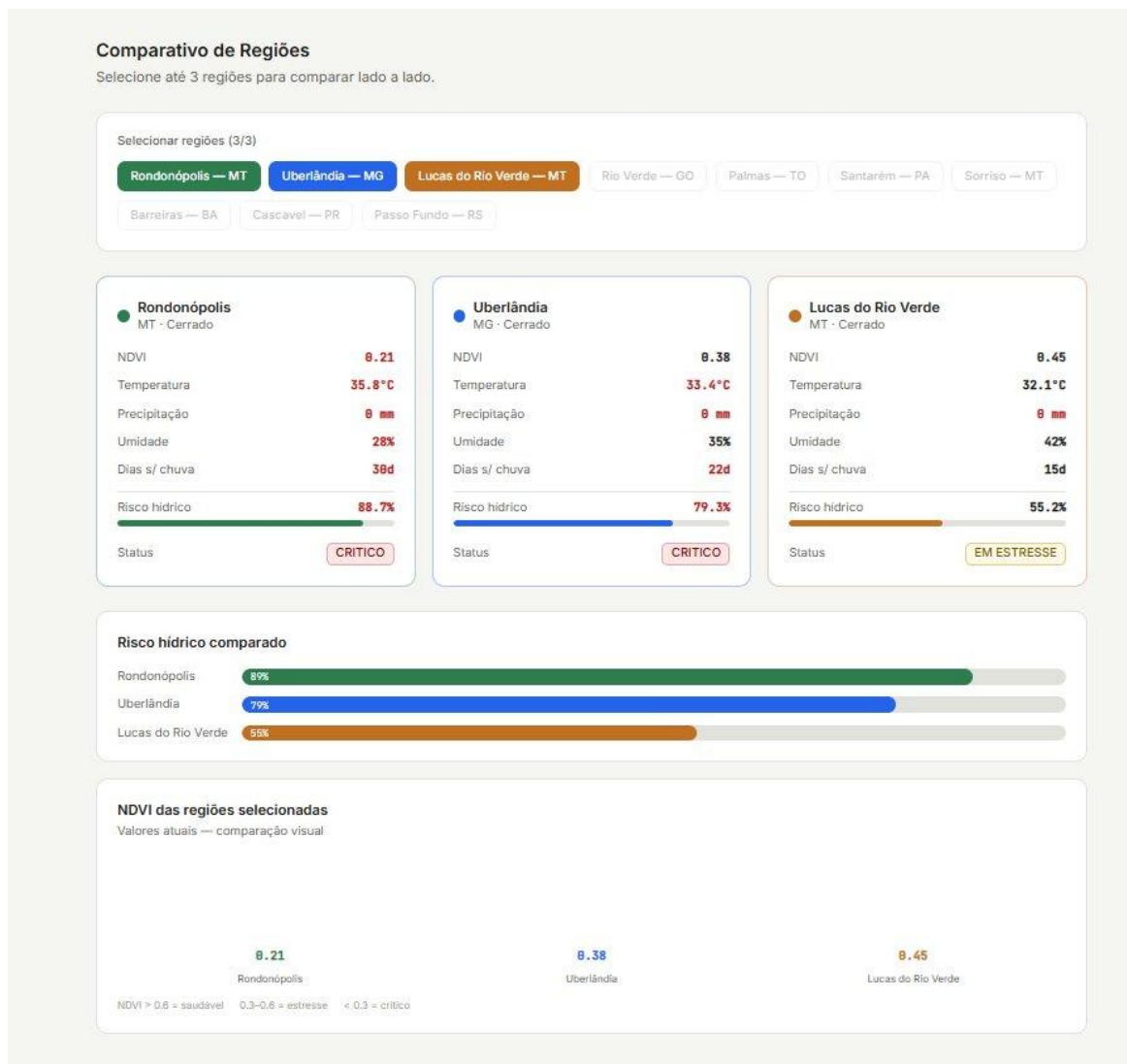


Figura 4 — Comparativo de Regiões: GET /previsoes/regiao/{id} e GET /leituras/regiao/{id}

7. TABELA DE ENDPOINTS

7.1 Health Check

Verbo	URI	Descrição	Status
GET	/health	Verifica status da API e conexão Oracle	200

7.2 Fontes Satelitais

Verbo	URI	Descrição	Status
GET	/fontes	Listar todas as fontes	200
GET	/fontes/{id}	Buscar fonte por ID	200 / 404
POST	/fontes	Cadastrar nova fonte	201 / 400
PUT	/fontes/{id}	Atualizar fonte	200 / 404
DELETE	/fontes/{id}	Deletar fonte	204 / 404

7.3 Regiões

Verbo	URI	Descrição	Status
GET	/regioes	Listar todas as regiões	200
GET	/regioes/{id}	Buscar região por ID	200 / 404
GET	/regioes/estado/{estado}	Listar regiões por estado (sigla)	200 / 400
GET	/regioes/situacao/cep/cep}	Busca por CEP com dados agrícolas do banco	200 / 404
POST	/regioes	Cadastrar região manualmente	201 / 400
POST	/regioes/buscar-por-cidade	Cadastrar com coords automáticas (cidade+estado)	201 / 404
PUT	/regioes/{id}	Atualizar região	200 / 404
DELETE	/regioes/{id}	Deletar região	204 / 404

7.4 Usuários

Verbo	URI	Descrição	Status
GET	/usuarios	Listar todos os usuários	200
GET	/usuarios/{id}	Buscar usuário por ID	200 / 404
POST	/usuarios	Cadastrar usuário	201 / 400 / 409
PUT	/usuarios/{id}	Atualizar usuário	200 / 404
DELETE	/usuarios/{id}	Deletar usuário	204 / 404

7.5 Leituras Satelitais

Verbo	URI	Descrição	Status
GET	/leituras	Listar todas (últimas 50)	200
GET	/leituras/{id}	Buscar leitura por ID	200 / 404
GET	/leituras/regiao/{idRegiao}	Listar leituras de uma região	200 / 404
POST	/leituras	Cadastrar leitura manual + IA	201 / 400
POST	/leituras/automatico/{idRegiao}	Coletar dados NASA+Meteo+IA e salvar	201 / 404
DELETE	/leituras/{id}	Deletar leitura (cascata)	204 / 404

7.6 Previsões

Verbo	URI	Descrição	Status
GET	/previsoes	Listar todas (últimas 50)	200
GET	/previsoes/{id}	Buscar previsão por ID	200 / 404
GET	/previsoes/regiao/{idRegiao}	Listar previsões de uma região	200
POST	/previsoes	Cadastrar previsão manualmente	201 / 400

7.7 Alertas

Verbo	URI	Descrição	Status
GET	/alertas	Listar todos (por criticidade)	200
GET	/alertas/pendentes	Listar alertas não resolvidos	200
GET	/alertas/regiao/{idRegiao}	Listar alertas de uma região	200
GET	/alertas/{id}	Buscar alerta por ID	200 / 404
GET	/alertas/{iid}/historico	Histórico de ações do alerta	200 / 404
PUT	/alertas/{iid}/esolver	Marcar alerta como resolvido	200 / 400 / 404
POST	/alertas/historico	Registrar ação no histórico	201 / 400
DELETE	/alertas/{id}	Deletar alerta	204 / 404

7.8 Regiões × Usuários

Verbo	URI	Descrição	Status
GET	/regioes-usuarios/{usuario/{id}}	Listar regiões de um usuário	200 / 404
GET	/regioes-usuarios/regiao/{id}	Listar usuários de uma região	200 / 404
POST	/regioes-usuarios	Vincular região a usuário	201 / 409
DELETE	/regioes-usuarios/{id}	Remover vínculo	204 / 404

8. INSTRUÇÕES DE EXECUÇÃO

8.1 Execução Local

Pré-requisitos: Java 21, Maven 3.9+ e acesso à rede FIAP (Oracle).

Passo	Comando / Ação
1. Clonar o repositório	<code>git clone https://github.com/GustavoSiciliano/Global-Solution-2026</code>
2. Acessar a pasta Java	<code>cd Global-Solution-2026/Java</code>
3. Iniciar o servidor	<code>mvn quarkus:dev</code>
4. Verificar	Acessar <code>http://localhost:8080/health</code> no navegador

8.2 MainTeste — Terminal Interativo

Passo	Comando
1. Compilar	<code>mvn compile</code>
2. Rodar local	<code>java -cp target/classes AgroSat.Teste.MainTeste http://localhost:8080</code>
3. Rodar Render	<code>java -cp target/classes AgroSat.Teste.MainTeste https://global-solution-2026.onrender.com</code>

8.3 Deploy no Render

Passo	Ação
1. GitHub	Subir o projeto na branch main
2. Render	New → Web Service → selecionar repositório
3. Configurar	Runtime: Docker Root Directory: Java Branch: main
4. Deploy	Clicar em 'Deploy Web Service' build ~10 minutos
5. Verificar	Acessar <code>/health</code> e confirmar oracle: conectado

9. FERRAMENTAS E VERSÕES

Tecnologia	Versão	Uso
Java JDK	21	Linguagem principal
Quarkus	3.8.4	Framework REST
Oracle JDBC	ojdbc11 23.3.0	Driver de conexão Oracle
Jackson	incluído no Quarkus	Serialização JSON
Maven	3.9+	Build e dependências
Docker	multi-stage	Containerização para-Render
IntelliJ IDEA	2024+	IDE recomendada
Oracle Database	FIAP — orcl	oracle.fiap.com.br:1521:orcl

10. LINKS DO PROJETO

Item	Link
Deploy (Render)	https://global-solution-2026.onrender.com
Health Check	https://global-solution-2026.onrender.com/health
GitHub	https://github.com/GustavoSiciliano/Global-Solution-2026
Vídeo Demonstração (YouTube)	https://youtu.be/tiokmaJ4eYg
API Flask IA	https://gs-ia.onrender.com

11. CONCLUSÃO

O AgroSat Java API representa uma solução backend completa e funcional para monitoramento agrícola inteligente. A API expõe 39 endpoints REST distribuídos em 8 recursos, cobre todas as tabelas do banco de dados Oracle e integra automaticamente três fontes de dados satelitais (NASA POWER, Open-Meteo e INPE TerraBrasilis) com um modelo de Machine Learning em produção.

A arquitetura em camadas (VO, DAO, BO, API) garante separação clara de responsabilidades, enquanto o tratamento centralizado de exceções assegura respostas JSON padronizadas em qualquer cenário de erro. O deploy no Render com Docker multistage torna a API disponível publicamente para integração com o frontend React.

O projeto atende a todos os requisitos da disciplina: CRUD completo, camadas obrigatórias, CORS configurado, tratamento de exceções, padrão DAO/BO, deploy em produção e integração com o frontend React do AgroSat.